

SMART LABORATORY ENVIRONMENT MONITORING SYSTEM

An IoT Based Embedded System with AI Driven Environmental Safety Analysis

Embedded Systems Project Report

Submitted by

M.A. KAUSHIK (CH.SC.U4CSE24123)

DEEPAK SN (CH.SC.U4CSE24112)

Department of Computer Science and Engineering

Institution: _____

Project guide: _____

Academic year: _____

Abstract

Laboratories must keep temperature, humidity and air quality within safe limits to protect people, experiments and expensive equipment. Manual monitoring is slow, and hazardous conditions are often noticed only after they have developed. This project presents a Smart Laboratory Environment Monitoring System built around an ESP32 microcontroller with a DHT22 (temperature and humidity), MQ-135 (air quality), PIR (occupancy) and LDR (ambient light) sensor. The system shows live readings on an OLED display, classifies the laboratory as SAFE, WARNING or CRITICAL, and automatically drives an exhaust fan (through a relay), a buzzer and status LEDs. Readings are published over Wi-Fi using MQTT to a web dashboard, stored in a cloud database (ThingSpeak and an SQLite logger) and critical events are sent as Telegram notifications.

The distinguishing feature is an AI safety predictor that runs on the ESP32 itself (TinyML). A small neural network (8-16-8-3, 307 parameters, 1228 bytes) uses the current readings together with their 10-minute trends to predict the worst laboratory condition in the next 10 minutes. On held-out test data it reached 97.6 % accuracy and 95.9 % recall for CRITICAL conditions, and it predicted all 22 critical episodes in the test period in advance, with a median lead time of 9.5 minutes, whereas a conventional threshold system reacts only at the moment the limit is crossed. The early warning starts ventilation before the hazard occurs. The complete system was developed and verified in software - Wokwi ESP32 simulation, a browser dashboard running the same model, a Python data pipeline and a Blender 3D digital twin - so that it can be moved to hardware with minimal change.

Keywords: IoT, ESP32, TinyML, environmental monitoring, laboratory safety, MQTT, predictive analytics, digital twin.

Table of Contents

1. Introduction
2. Target Users and Their Challenges
3. Literature Survey
4. Methodology
5. Components and Specifications
6. System Architecture
7. Hardware Design
8. Firmware Design
9. IoT, Cloud and Dashboard
10. Machine Learning and TinyML
11. Simulation, Digital Twin and Testing
12. Results
13. Advantages, Limitations and Future Scope
14. Conclusion

References

Appendix A - Project Files

Appendix B - Moving to Hardware

(In Word: References -> Table of Contents replaces this list with an automatic one.)

1. Introduction

1.1 Problem

Many laboratories struggle to maintain safe environmental conditions during daily operations. Excess heat damages instruments and affects experiments, high humidity corrodes equipment and spoils samples, and poor air quality - carbon dioxide from occupants or fumes from solvents - affects health and concentration. These conditions build up gradually and are usually detected late, by people, if at all.

1.2 Solution

We designed an intelligent monitoring system for laboratories and research facilities that measures the environment continuously, detects unsafe conditions automatically, responds by ventilating and alerting, stores data for analysis, and uses machine learning to predict unsafe conditions before they occur.

1.3 Why it matters

Making laboratory monitoring intelligent and automated improves safety, protects equipment, supports compliance with laboratory safety practice and gives facility managers historical data to plan maintenance.

1.4 Objectives

1. Monitor laboratory environmental parameters continuously.
2. Detect unsafe conditions automatically.
3. Alert users through alarms and IoT notifications.
4. Store sensor data for future analysis.
5. Integrate machine learning to predict unsafe conditions before they occur.

1.5 Action plan and status

#	Phase	Status
1	Literature survey	Completed (Phase 1)
2	Component selection	Completed (Phase 1)
3	Hardware design	Completed - circuit in Wokwi (diagram.json), pin map in Section 7
4	Sensor integration	Completed in simulation (DHT22, MQ-135, PIR, LDR)
5	Embedded programming (ESP32)	Completed - firmware/sketch.ino
6	IoT dashboard development	Completed - dashboard/index.html (MQTT, history, controls)
7	Data collection	Pipeline completed (cloud/mqtt_logger.py, ThingSpeak); synthetic dataset used until hardware is built
8	ML model development	Completed - ml/train_model.py

9	Deploy model on embedded board	Completed - model.h runs on the ESP32 (TinyML)
10	Testing and validation	Completed in software (Section 11); hardware validation pending

2. Target Users and Their Challenges

Laboratory technicians, researchers, faculty members and students who work in laboratories that require safe and controlled environmental conditions.

Challenge	How the system addresses it
Manual monitoring is time-consuming	Sensors are read every second, 24/7, with no human effort
Unsafe temperature, humidity or air quality damages equipment and experiments	Three-level status with automatic ventilation; AI starts ventilation before limits are crossed
Delayed detection increases safety risk	Immediate local alarm plus Telegram and dashboard alerts
No real-time alerts or historical data	Live dashboard, cloud database, trend charts and CSV export

3. Literature Survey

Existing approaches to laboratory and indoor environment monitoring were reviewed.

Approach	Strength	Gap addressed here
Stand-alone thermo-hygrometers and CO2 meters	Simple, cheap	No alerts, no logging, no automation
Building Management Systems (BMS)	Centralised HVAC control	Expensive; rarely room-level air quality or occupancy
Arduino/ESP32 IoT monitors (Blynk, ThingSpeak)	Low cost, remote viewing	Mostly threshold alarms; react only after a limit is crossed
Cloud ML analytics on sensor data	Can find patterns	Needs internet; latency; no action on the device
TinyML on microcontrollers	Inference on the device, offline	Rarely applied to lab safety prediction

Reference limits used in this project are drawn from common indoor-environment guidance: comfortable laboratory temperature of about 18-27 C, relative humidity of 30-60 %, and indoor CO2 below about 1000 ppm as a ventilation indicator. These values are configurable in the firmware.

4. Methodology

Stage	Work done
Research	Studied laboratory safety practice and existing monitoring systems
System design	Selected sensors (temperature, humidity, air quality, occupancy, light) and actuators
Prototyping	Firmware for real-time acquisition, processing and actuator control; Wokwi circuit
AI development	Dataset, feature engineering (trends), model comparison, export to C for the ESP32
Testing and validation	Model verification, dashboard scenario tests, cloud pipeline tests, simulation
Final build	Hardware assembly using the same firmware (Appendix B)

5. Components and Specifications

Component	Purpose	Specification
ESP32 DevKit-C	Main controller	Dual-core 240 MHz, Wi-Fi and Bluetooth, 3.3 V logic, 12-bit ADC
DHT22	Temperature and humidity	-40 to 80 C (+/-0.5 C), 0-100 % RH (+/-2 %)
MQ-135	Air quality	Detects CO ₂ , NH ₃ , smoke and VOCs; analog output; 5 V heater
PIR sensor (HC-SR501)	Occupancy detection	Motion detection, range 5-7 m
LDR module	Ambient light	Light-dependent resistor with 10 kohm divider, lux estimate
OLED display	Live monitoring	0.96 inch, 128 x 64, SSD1306, I2C (0x3C)
Relay module	Fan control	5 V coil, 10 A contacts
Exhaust fan	Automatic ventilation	Switched by the relay on unsafe or predicted-unsafe conditions
Buzzer, LEDs	Audio and visual alerts	Green SAFE, yellow WARNING / early warning, red CRITICAL
Push button	User input	Short press: mute 60 s; long press: self-test
Wi-Fi	Cloud communication	MQTT (live), HTTP (ThingSpeak), HTTPS (Telegram)

6. System Architecture

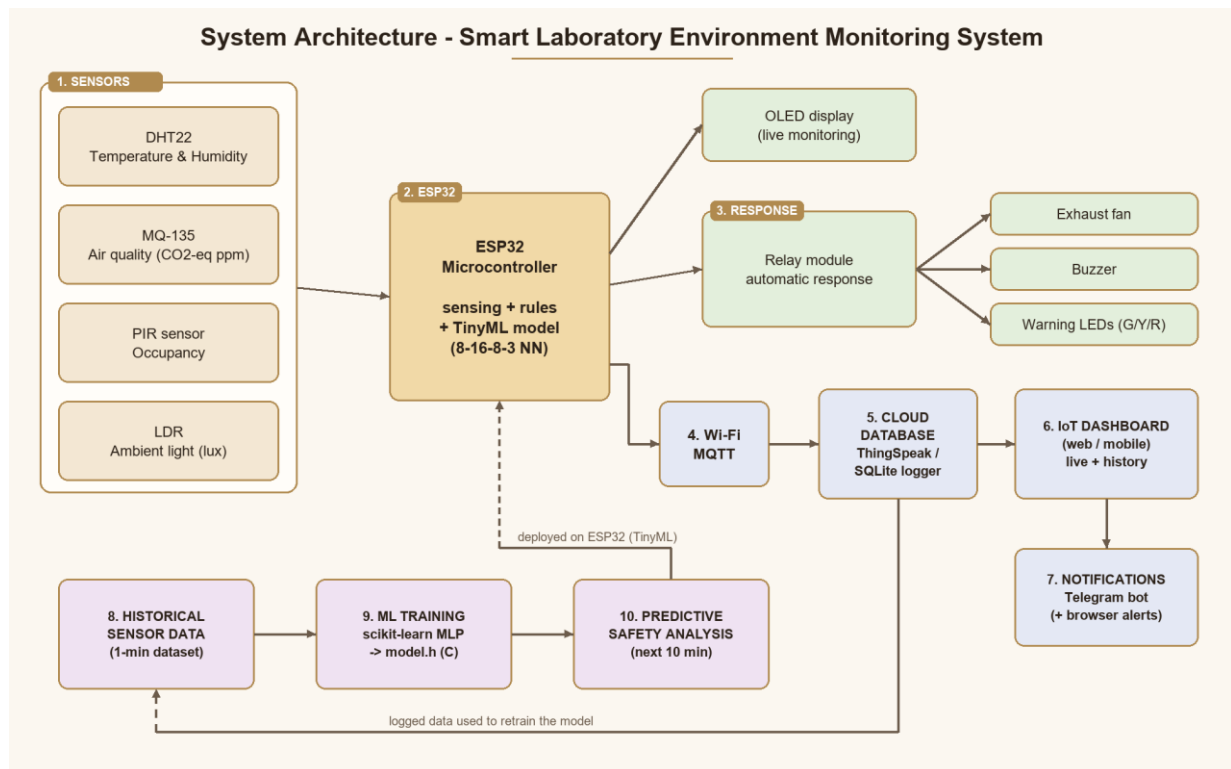


Figure 6.1 - System architecture

The ten blocks follow the Phase 1 architecture: (1) sensors feed (2) the ESP32, which drives (3) the automatic response and the OLED; data flows over (4) Wi-Fi to (5) the cloud database, (6) the IoT dashboard and (7) notifications. (8) Historical data is used to (9) train the ML model, whose (10) predictive safety analysis runs back on the ESP32. Logged real data closes the loop for retraining.

7. Hardware Design

7.1 Pin assignment

ESP32 pin	Connected to	Type
GPIO 15	DHT22 data	Digital (1-wire protocol)
GPIO 34	MQ-135 AO	Analog input (ADC1)
GPIO 35	LDR module AO	Analog input (ADC1)
GPIO 27	PIR OUT	Digital input
GPIO 21 / 22	OLED SDA / SCL	I2C
GPIO 26	Relay IN (exhaust fan)	Digital output
GPIO 13	Buzzer	PWM tone
GPIO 25 / 33 / 32	Green / yellow / red LED (220 ohm)	Digital output
GPIO 4	Push button to GND	Digital input, internal pull-up
5V / 3V3 / GND	MQ-135 and relay (5 V); DHT22, PIR, LDR, OLED (3.3	Power

	V)	
--	----	--

7.2 Circuit notes

- Only ADC1 pins (32-39) are used for analog sensors, because ADC2 cannot be used while Wi-Fi is active.
- The MQ-135 is powered from 5 V; on hardware its analog output must pass through a 10 k / 20 k divider to stay below 3.3 V (firmware constant DIVIDER = 1.5).
- The relay module drives the fan; in the simulation a blue LED on the relay contact represents the fan.
- The complete wiring is in firmware/diagram.json and opens directly in Wokwi.

8. Firmware Design

8.1 Status rules (deterministic safety layer)

Status	Condition	Response
CRITICAL	T > 32 or < 15 C; RH > 70 or < 20 %; air > 2000 ppm	Blinking red LED, two-tone siren, fan ON, Telegram + MQTT alert
WARNING	T > 27 or < 18 C; RH > 60 or < 30 %; air > 1000 ppm; occupied with light < 150 lux	Yellow LED; fan ON when caused by air, heat or humidity; alert
SAFE	All parameters in range	Green LED; fan in automatic mode (off)
AI EARLY WARNING	Model predicts CRITICAL within 10 min with $p \geq 0.6$ while not yet critical	Fan ON pre-emptively, blinking yellow LED, double chirp, Telegram alert

To avoid alarm chattering when a reading hovers near a limit, the status rises immediately but falls only after it has stayed lower for five model-minutes (time hysteresis). The same rules produce the training labels for the ML model, so the AI and the rule layer are consistent.

8.2 Sensor processing

MQ-135: $R_s = R_L (V_c - V_{out}) / V_{out}$, $ppm = 116.602 (R_s/R_0)^{-2.769}$ (CO2 curve)
 R_0 calibrated at start-up in fresh air (420 ppm); EMA filter 0.7/0.3
LDR: $R = 2000 V / (1 - V/3.3)$, $lux = (R/L10 * 1e3 * 10^{0.7} / R)^{(1/0.7)}$
PIR: occupied = motion within the last 60 s (5 min on hardware)
Trends: 10-sample history -> slope = (newest - oldest) / 10 per model-minute

8.3 Program structure

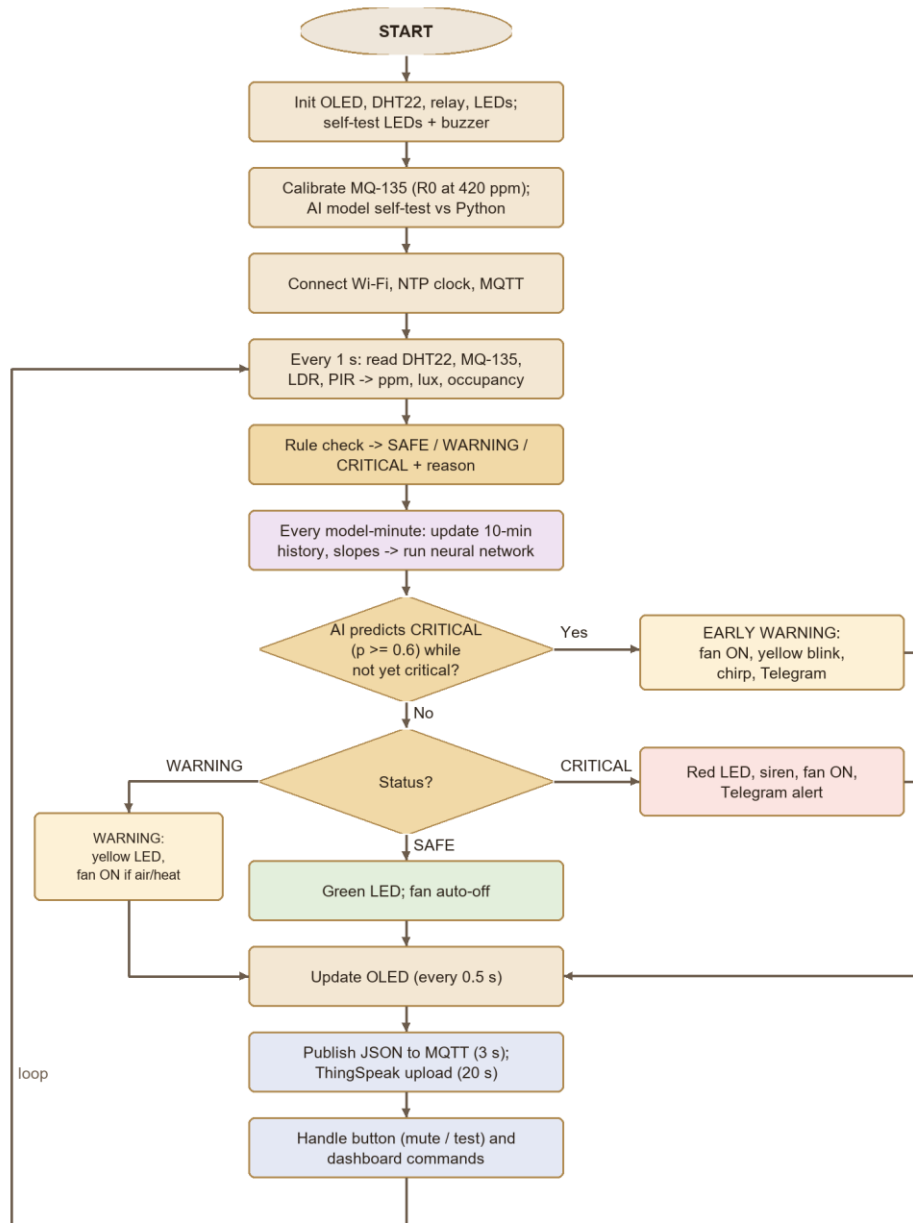


Figure 8.1 - Firmware flowchart

Function	Purpose	Period
readSensors()	DHT22, MQ-135, LDR, PIR -> engineering units	1 s (DHT22 every 2 s)
evaluateStatus()	Rule-based status and reason text	1 s
updateModel()	Trend history, slopes, neural-network inference	1 model-minute
applyOutputs()	LEDs, relay (fan), buzzer patterns	every loop
updateOled()	OLED screen with clock, readings, status, message	0.5 s
publishData()	JSON status to MQTT	3 s
uploadCloud()	ThingSpeak fields 1-8	20 s
checkNotifications()	Telegram / MQTT alerts on escalation, early warning,	event

	recovery	
onCommand()	Dashboard commands: fan_on, fan_auto, mute, test	event

All timing uses millis() so that sensing, control, display and networking run concurrently, and the safety logic keeps working even if Wi-Fi or the cloud is unavailable. MODEL_MINUTE_MS is 60 000 ms on hardware and 3 000 ms in the simulator so that trends develop quickly during a demonstration.

9. IoT, Cloud and Dashboard

9.1 MQTT topics

Topic	Direction	Content
smartlab/<id>/data	ESP32 -> dashboard / logger	JSON every 3 s (last-will 'offline')
smartlab/<id>/alert	ESP32 -> dashboard / logger	Alert messages (status changes, AI early warning)
smartlab/<id>/cmd	dashboard -> ESP32	fan_on fan_auto mute test

```
{ "temp":28.9, "hum":53, "aq":979, "aqLabel":"Moderate", "occ":true, "lux":509, "status":"WARNING",
  "reason":"Temperature abnormal", "pred":"CRITICAL", "pSafe":0.0, "pWarn":0.15, "pCrit":0.85,
  "early":true, "slopeT":0.30, "fan":true, "buzzer":true, "cloud":true, "rssi":-58, ... }
```

9.2 Cloud database

- **ThingSpeak:** the ESP32 uploads eight fields every 20 s (temperature, humidity, air ppm, occupancy, lux, status, AI class, P(critical)); the dashboard can load this history.
- **SQLite logger:** cloud/mqtt_logger.py subscribes to MQTT and stores every reading and alert; '-export' produces a one-minute dataset with the same columns as the training data.

9.3 Notifications

Telegram messages are sent by the ESP32 over HTTPS for escalations, AI early warnings and recovery, rate-limited to one per minute except for escalations. The dashboard additionally plays an alarm and shows a browser notification on CRITICAL.

9.4 Web / mobile dashboard

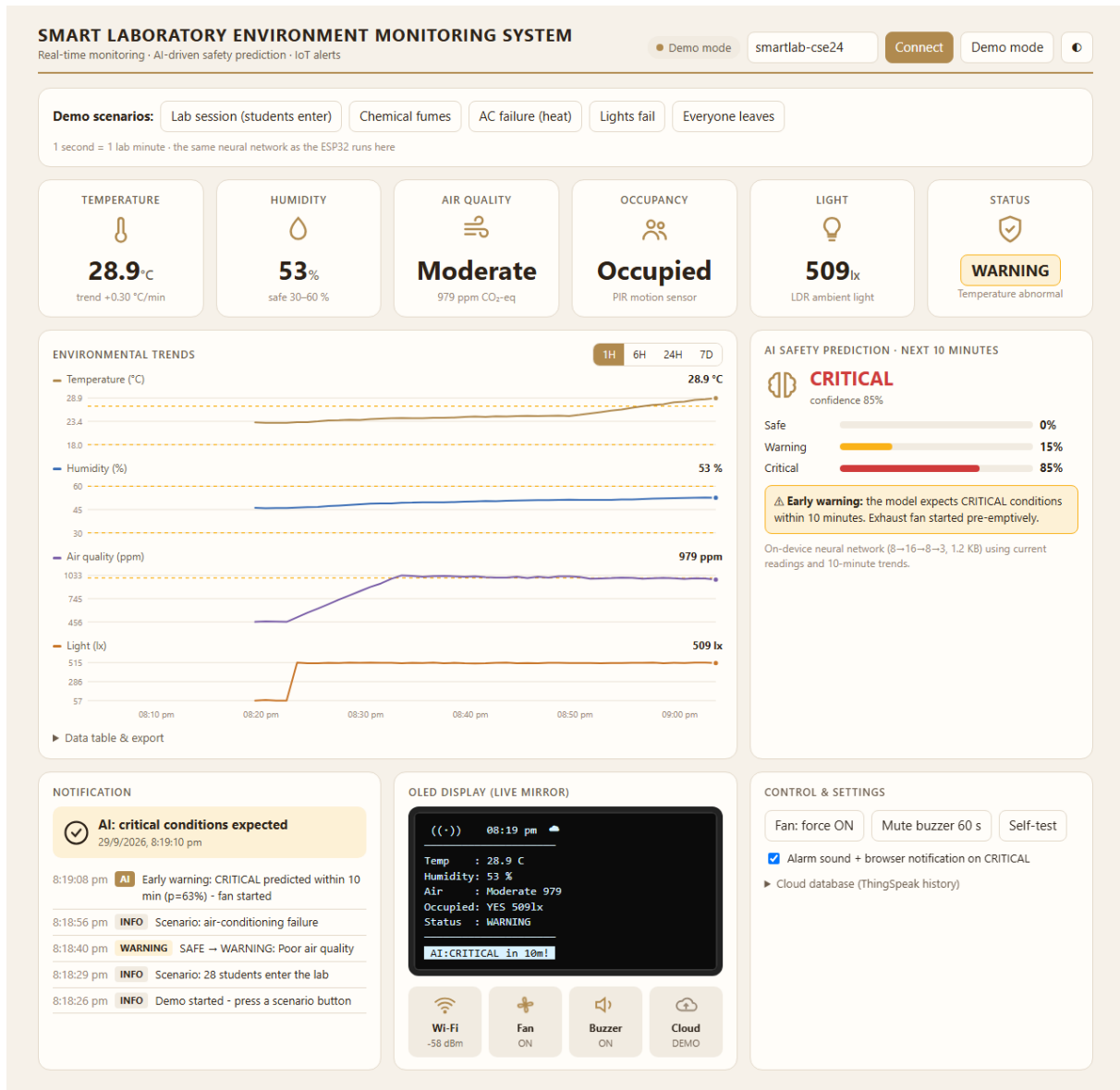


Figure 9.1 - Dashboard during the AC-failure scenario: the AI predicts CRITICAL (85 %) while the lab is still at WARNING and has started the fan

The dashboard reproduces the Phase 1 expected output: cards for temperature, humidity, air quality, occupancy, light and status; environmental trend charts with 1H / 6H / 24H / 7D ranges (drawn as separate small charts rather than a dual-axis chart so each quantity keeps its own scale); a notification panel and event log; a live mirror of the OLED; system indicators for Wi-Fi, fan, buzzer and cloud; the AI prediction with class probabilities; remote controls; CSV export; and loading of ThingSpeak history. Demo mode runs a lab simulation with the same neural network, for presentations without the simulator.

10. Machine Learning and TinyML

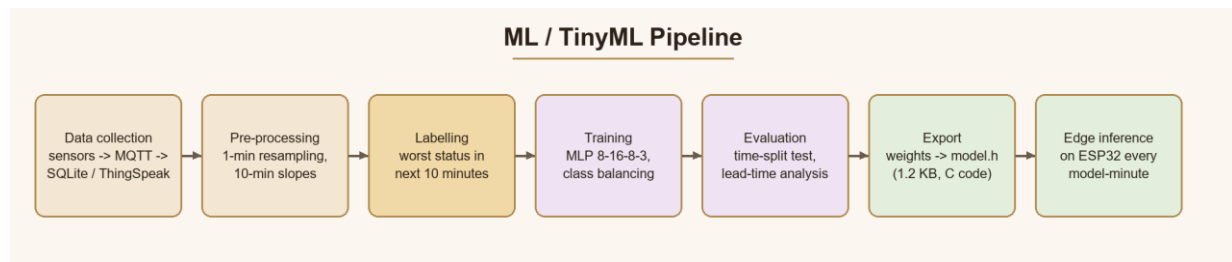


Figure 10.1 - ML pipeline

10.1 Dataset

Until the hardware is installed, a physically motivated synthetic dataset was generated (ml/generate_dataset.py): 90 days at one sample per minute (129,590 rows). It models a lab timetable, CO2 generation by occupants and removal by natural and fan ventilation, HVAC set-point control with random air-conditioning failures, humidity events, solvent releases and lamp failures, with sensor noise. The fan is driven by the same rule controller as the firmware. The pipeline is designed so that real data logged by the system replaces the synthetic data without code changes.

10.2 Features and target

Feature	Meaning
temperature, humidity, air_quality_ppm, occupancy, light_lux	Current (filtered) sensor values
temp_slope, aq_slope, hum_slope	Change per minute over the last 10 minutes (trend)
Target: status_next	Worst status (SAFE / WARNING / CRITICAL) in the next 10 minutes

The data was split by time: days 0-69 for training and the remaining days for testing, so the test measures performance on unseen future days. Minority classes were over-sampled in training.

10.3 Model comparison (test days)

Model	Accuracy	Macro F1	CRITICAL recall	CRITICAL precision
Logistic Regression	95.1 %	0.909	86.8 %	87.9 %
Decision Tree (depth 8)	98.0 %	0.956	94.9 %	90.5 %
Neural Network MLP (16, 8)	97.6 %	0.941	95.9 %	83.4 %
Threshold rules (current reading only)	92.6 %	0.881	81.5 %	100.0 %

The neural network was selected for deployment because it gives the highest recall for CRITICAL conditions (95.9 %) - missing a hazard is worse than a false alarm - while remaining tiny (1228

bytes). The decision tree has slightly higher overall accuracy and is a good alternative. The threshold baseline can only report the present state, so its 'predictions' miss conditions that are about to develop.

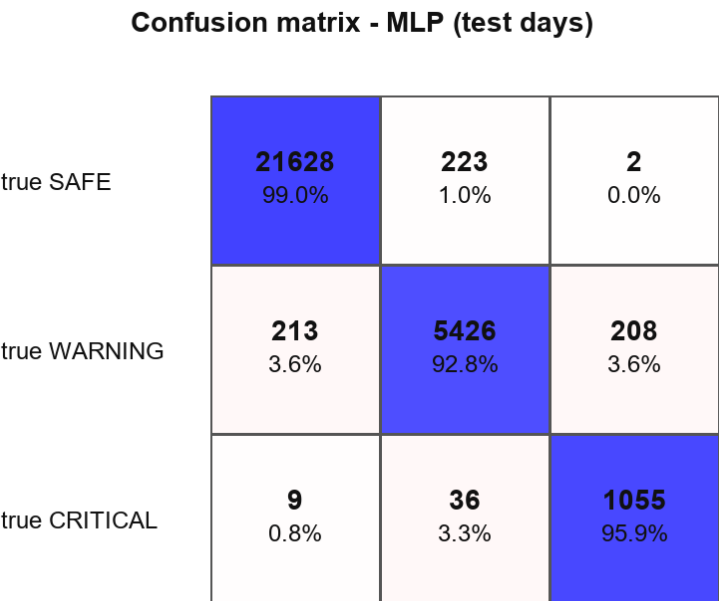


Figure 10.2 - Confusion matrix of the deployed neural network

10.4 Early-warning performance

Metric	Threshold rules	AI model
Critical episodes in test period	22	22
Predicted before onset	0 %	100 %
Median warning lead time	0 min	9.5 min

A lead time of several minutes is enough for the exhaust fan to bring air quality or temperature back before a critical level is reached, which the demo scenarios show (Section 11). Because the dataset is synthetic, these figures show that the method works; they must be re-measured on real laboratory data.

10.5 Deployment on the ESP32 (TinyML)

Neural network deployed on the ESP32

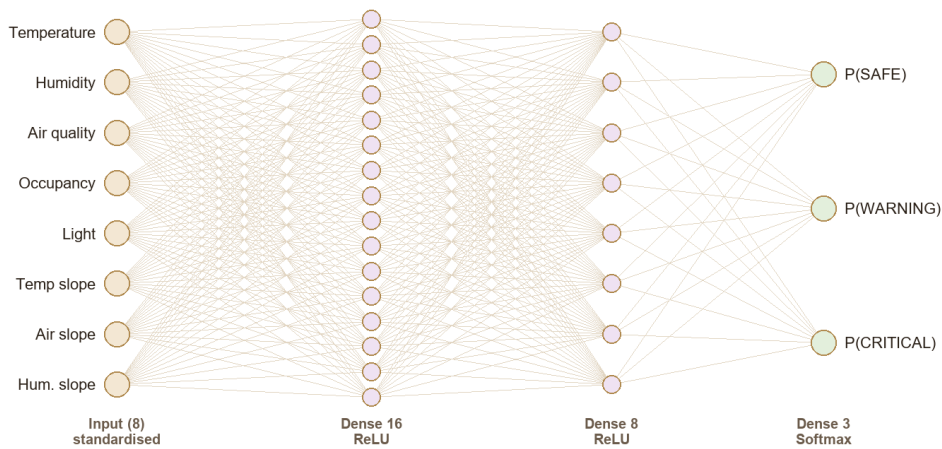


Figure 10.3 - Deployed network

`train_model.py` exports the standardisation constants and all weights as C arrays in `model.h`, together with a forward-pass function (dense layers, ReLU, softmax) and three self-test vectors. At start-up the ESP32 checks that its outputs match Python's; in a desktop g++ test the maximum difference was $4.8e-7$. Inference needs about 300 multiply-adds, a few microseconds on the ESP32, and no external library, which is the same principle as TensorFlow Lite for Microcontrollers with a smaller footprint. The same weights are exported to `dashboard/ml_model.js` so the dashboard can display and simulate identical predictions.

11. Simulation, Digital Twin and Testing

11.1 Wokwi simulation

The complete project (firmware, AI model, circuit and libraries) is saved on Wokwi at <https://wokwi.com/projects/476557350477755393>. To run it:

1. Open the project link (or run `start_all.bat` wokwi, which also starts the logger and dashboard).
2. To rebuild it elsewhere: paste `sketch.ino` and `diagram.json` into a new ESP32 project, add `model.h` and `libraries.txt` (DHT sensor library for ESPx, Adafruit SSD1306, Adafruit GFX Library, PubSubClient).
3. Start the simulation. The OLED shows readings; the serial monitor prints the AI prediction each model-minute.
4. Click the DHT22 to change temperature/humidity, the gas sensor to change air quality, the LDR to change light, and 'Simulate motion' on the PIR for occupancy.
5. Open `dashboard/index.html` and press Connect to watch the simulated device live.

11.2 Blender digital twin

blender/smart_lab_twin.py builds a 3D laboratory - benches, fume hood, air-conditioner, exhaust fan, ceiling lights, students and the monitoring node with a working OLED - and animates the AI early-warning story: students enter, CO2 rises (shown as haze), the model predicts CRITICAL, the fan starts, a Telegram message appears, and the air returns to SAFE without ever reaching CRITICAL. A wall dashboard and the OLED show changing values every frame. With LIVE_MQTT enabled it mirrors the real (or simulated) device in real time - the digital twin listed as future scope in Phase 1.

11.3 Test cases

ID	Test	Expected result	Observed
T1	Power-on	LED sequence + chirp, MQ-135 calibration, AI self-test PASSED in serial	
T2	Normal conditions (24 C, 45 %, 400 ppm)	SAFE, green LED, fan off, OLED 'Environment Stable'	
T3	Air quality raised above 1000 ppm	WARNING 'Poor air quality', yellow LED, fan ON	
T4	Temperature set to 35 C	CRITICAL, blinking red LED, siren, fan ON, alert	
T5	PIR motion with light < 150 lux	WARNING 'Insufficient light'	
T6	Temperature raised steadily (trend)	AI early warning before 32 C: fan ON, OLED 'AI:CRITICAL in 10m!'	
T7	Button short press / long press	Buzzer muted 60 s / self-test sequence	
T8	Dashboard connected to Wokwi	Live values every 3 s; commands executed	
T9	Model C code vs Python	Identical probabilities on test vectors	Pass - C inference = Python (max error 4.8e-7, g++ test)
T10	Dashboard demo: lab session	WARNING at ~1000 ppm and ventilation	Pass - WARNING at ~1000 ppm, fan ON, CO2 held near 1000 ppm
T11	Dashboard demo: AC failure	AI early warning before CRITICAL	Pass - AI early warning ~13 lab-minutes before CRITICAL
T12	MQTT logger end-to-end	Readings and alerts stored; training CSV exported	Pass - 40 readings + 1 alert stored; CSV export OK
T13	Reading hovering near a limit	No repeated SAFE/WARNING alerts (hold time)	Pass - no SAFE/WARNING flapping after hold-time fix

T1-T8 are completed in the Wokwi simulator (insert screenshots here, Figures 11.1-11.4). T9-T13 were executed during development and passed.

12. Results

- Continuous monitoring of five parameters with a three-level status and automatic ventilation.
- AI prediction on the ESP32: 97.6 % accuracy, 95.9 % CRITICAL recall, all 22 test critical episodes predicted in advance (median 9.5 min).
- In the AC-failure scenario the early warning came about 13 lab-minutes before the CRITICAL limit; in the lab-session scenario the fan kept CO₂ near 1000 ppm instead of letting it climb.
- Live dashboard, cloud logging, Telegram alerts and a 3D digital twin complete the IoT chain.
- Model size 1228 bytes - well within the ESP32's 520 KB RAM and 4 MB flash.

13. Advantages, Limitations and Future Scope

13.1 Advantages

- Continuous monitoring and automated alerts
- Remote access from any browser or phone
- Low-cost implementation with common modules
- Scalable to many labs (one device ID per room)
- AI-based predictive safety running offline on the device
- Safety layer independent of the network

13.2 Limitations

- The ML model is trained on synthetic data; it must be retrained on real logged data.
- MQ-135 gives a CO₂-equivalent estimate, is cross-sensitive to other gases and drifts; it needs burn-in and periodic calibration (an NDIR CO₂ sensor or BME680 would be more accurate).
- A PIR sensor detects presence, not the number of people.
- The public MQTT broker is for demonstration; a private broker with authentication and TLS is needed for deployment.

13.3 Future scope

- Computer vision for PPE detection
- Voice alerts
- Digital twin of the laboratory (prototype built in Blender)
- Predictive maintenance of HVAC
- Integration with Building Management Systems (BMS)
- Edge AI using TinyML - extend to anomaly detection and sensor-fault detection
- People counting with door sensors or thermal array sensors
- Email / SMS alerts and multi-lab dashboard

14. Conclusion

The Smart Laboratory Environment Monitoring System provides a reliable and cost-effective way to improve laboratory safety through continuous monitoring of temperature, humidity, air quality, occupancy and light. By combining an ESP32-based embedded system with IoT

communication, cloud storage and an on-device neural network, it offers real-time monitoring, automated alerts, data logging and intelligent ventilation control - and, beyond conventional threshold systems, it predicts unsafe conditions minutes before they occur and acts on them. The complete design was implemented and verified in software, and the same firmware is ready for the hardware build.

References

1. Espressif Systems, 'ESP32 Series Datasheet' and 'ESP32 Technical Reference Manual'.
2. Aosong Electronics, 'AM2302 / DHT22 Digital Temperature and Humidity Sensor' datasheet.
3. Hanwei Electronics, 'MQ-135 Gas Sensor' technical data.
4. Solomon Systech, 'SSD1306 128x64 OLED Driver' datasheet.
5. ASHRAE Standard 62.1, 'Ventilation for Acceptable Indoor Air Quality'.
6. P. Warden and D. Situnayake, 'TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers', O'Reilly, 2019.
7. F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python', JMLR 12, 2011.
8. OASIS, 'MQTT Version 3.1.1', 2014.
9. Wokwi documentation, <https://docs.wokwi.com>; ThingSpeak documentation, <https://www.mathworks.com/help/thingspeak/>.
10. Telegram Bot API, <https://core.telegram.org/bots/api>; Blender Manual, <https://docs.blender.org>.

Appendix A - Project Files

Path	Contents
firmware/sketch.ino, model.h, diagram.json, libraries.txt	ESP32 firmware, AI model, Wokwi circuit
ml/generate_dataset.py, train_model.py	Dataset generation, training, evaluation, export
ml/metrics.json, confusion_matrix.png	Results used in this report
dashboard/index.html, ml_model.js	IoT dashboard with demo mode
cloud/mqtt_logger.py	MQTT -> SQLite logger and training-data export
blender/smart_lab_twin.py	3D digital twin and animation
report/	This report and the scripts that draw its figures

Appendix B - Moving to Hardware

1. Wire the circuit as in Section 7; add the 10 k / 20 k divider on MQ-135 AO and set DIVIDER = 1.5.
2. Set MODEL_MINUTE_MS = 60000 and OCCUPIED_HOLD_MS = 300000.
3. Burn in the MQ-135 for 24-48 h and calibrate it in fresh outdoor air.
4. Enter the lab Wi-Fi, a unique DEVICE_ID, the ThingSpeak write key and the Telegram bot token / chat id.
5. Run cloud/mqtt_logger.py for several weeks, export the data, retrain with ml/train_model.py and copy the new model.h into the firmware.
6. Mains-powered fans must be switched by a properly rated relay installed by a qualified electrician.